Bashir Agro Admin Panel
Phase 9: Settings & Audit Logs Module

Phase 9 Scope

Implement ONLY Settings and Audit Logs after Phase 8.

Included:
- System settings management
- Admin configuration panel
- Audit trail system
- User action tracking
- Historical change records
- Activity filtering
- Security tracking
- Tests

Not included:
- Flutter settings
- REST API settings
- Mobile configuration
- External integrations

Objective

Create an administrative control and accountability system.

Settings allow Admin/Owner to control configurable system values.

Audit Logs record important actions performed inside the Admin Panel.

The goal is:
- Better control
- Better security
- Complete historical tracking

Settings Architecture

Admin

↓

Settings Controller

↓

Settings Service

↓

Database

↓

Application Configuration

Settings should not be hard-coded inside controllers or Blade files.

Database Design: settings

Create:

settings

Fields:

id

key

value

type

group

description

is_editable

created_by

created_at

updated_at

Example:

key:
default_geofence_radius

value:
20

type:
integer

Setting Examples

Possible settings:

General:
- Farm name
- Company information

Task:
- Default task priority

- Default task duration

Location:
- Default geofence radius

System:
- Pagination size
- Date format

Keep settings flexible for future expansion.

Settings Model

Create:

app/Models/Setting.php

Requirements:

- Fillable protection
- Type casting
- Creator relationship

Example methods:

getValue()

setValue()

Settings Controller

Create:

app/Http/Controllers/Admin/SettingsController.php

Methods:

index()

update()

Responsibilities:

- Display settings
- Validate changes
- Save updates
- Create audit records

Settings Validation

Validate:

key

value

type

Rules:

Required fields

Correct data type

Allowed values where necessary

Settings UI

Create:

resources/views/settings/index.blade.php

Preserve:

- Bengali UI
- Green Bashir Agro theme
- Existing cards
- Existing forms
- Responsive design

Display:

Setting name

Description

Current value

Edit option

Audit Log Architecture

Every important Admin action creates an audit record.

Example:

Admin updated farmer phone number.

Record:

Who changed it?

What changed?

When?

Old value?

New value?

Database Design: audit_logs

Create:

audit_logs

Fields:

id

user_id

action

module

subject_type

subject_id

description

old_values

new_values

ip_address

user_agent

created_at

Audit Events

Track:

Farmer:
- Created
- Updated
- Deleted

Location:
- Created
- Updated
- Deleted

Task:
- Created
- Updated
- Deleted

Assignment:
- Created
- Status Changed

Settings:
- Updated

Audit Model

Create:

app/Models/AuditLog.php

Relationships:

belongsTo User

Use JSON storage for:

old_values

new_values

Audit Service

Recommended:

app/Services/AuditService.php

Responsibilities:

- Create logs
- Store changes
- Format descriptions
- Capture IP information

Controllers should call service instead of duplicating logic.

Audit Log Page

Create:

resources/views/audit/index.blade.php

Display:

Date

User

Module

Action

Description

IP Address

Details button

Audit Filters

Implement:

Date range

User filter

Module filter

Action filter

Example:

Show all farmer updates by Admin.

Search

Search by:

- User name
- Module
- Description
- Action

Example:

/admin/audit?search=farmer

Security Requirements

Implement:

- Authentication middleware
- Authorization
- Protected settings access
- Prevent unauthorized modifications
- Validate all inputs

Audit logs should not be editable or deletable by normal Admin users.

Performance

Use:

- Pagination
- Database indexes
- Efficient JSON queries
- Latest records first

Avoid loading complete audit history at once.

Routes

Add:

GET /admin/settings

PUT /admin/settings

GET /admin/audit

All routes must use Admin middleware.

Testing

Create:

tests/Feature/SettingsAuditTest.php

Test:

- Admin can view settings
- Admin can update settings
- Validation works
- Audit log created after changes
- Audit filtering works
- Unauthorized access blocked
- Pagination works

Antigravity Prompt

```
Proceed to PHASE 9 ONLY: Settings and Audit Logs Module.

Use the existing Bashir Agro Laravel project after Phase 8.

Do not redesign the UI.

Do not implement:

- Flutter settings
- REST API settings
- Mobile configuration
- External integrations


Implement:

- settings table
- Setting model
- Settings controller
- Settings UI
- Audit logs table
- AuditLog model
- Audit service
- Audit log page
- Filters
- Search
- Pagination
- Feature tests


Preserve:

- Bengali UI
- Green agricultural theme
- Existing cards
- Existing tables


Database must be the source of truth.

All important Admin actions must create audit records.


After implementation:

Run:

php artisan test

Run:

vendor/bin/pint


Report:

- Files created
```

```
- Files modified
- Database changes
- Routes added
- Test results


STOP after Phase 9.

Do not start Phase 10 automatically.
```